LABORATORY TITLE: HTTP ANALYSIS USING WIRESHARK - EMBEDDED IMAGE TRAFFIC

STUDENT NAME: WALIYAT OLALEKAN

STUDENT ID: 2025/FWSD/11435

COURSE NAME: BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

COURSE CODE: SBT-DF203 (LAB 2)

INSTRUCTOR NAME: MR. AMINU IDRIS

DATE OF SUBMISSION: SEPTEMBER 10TH, 2026.

VERSION: Version 1.0

Executive Summary

This forensic report covers the capture, TCP stream reassembly, object extraction, and hash verification of an unencrypted HTTP session transferring a web page containing an embedded image asset. The objective was to demonstrate how web browsers generate secondary HTTP requests upon parsing HTML DOM elements, reconstruct segmented TCP stream payloads, export file objects from raw packet captures, and verify cryptographic integrity against source files.

The scope of this investigation was restricted to local interactions between a web browser (Mozilla Firefox) and the local web daemon (Apache2) on loopback address 127.0.0.1.

Forensics Findings

Finding	Observation
Number of HTTP requests	2
HTML request URI and timestamp	GET /image.html at 21:19:32 GMT
Image request URI and timestamp	GET /lab_photo.jpg at 21:18:05 GMT
HTTP response codes	200 OK (for both)
Image content type	image/jpeg
Reported Content-Length	153 bytes - /image.html 1195963 bytes - /lab_photo.jpg
Number of reassembled TCP segments	4
Sum of TCP payload lengths	1846 bytes
Original image SHA-256	8091f1d7f30f652341834f83aec68ae5926caff9731e5e0a295678d351532c15 - /image.html c6e857a0b5364245b039896717093a3961abbf41a1de9f1e22c3bb08af7905a3 - /lab_photo.lab
Extracted image SHA-256	8091f1d7f30f652341834f83aec68ae5926caff9731e5e0a295678d351532c15 - /image.html c6e857a0b5364245b039896717093a3961abbf41a1de9f1e22c3bb08af7905a3 - /lab_photo.lab
Hash comparison conclusion	The extracted binary object is identical to the original file hosted on the server.
curl/browser difference	Browsers parse HTML DOM and automatically issue sub-resource GET requests for embedded assets (<img>, <script>). curl is a command-line HTTP client that fetches only the explicit URI requested (/image.html) and does not evaluate or render HTML tags.

Evidence Chain of Custody and Hash Records

File Description	Relative Path	File Type
Original capture	evidence/image_traffic.pcapng	pcapng capture
Working copy	working/image_traffic_working.pcapng	pcapng capture
Source HTML	/var/www/html/image.html	HTML document

Source image	/var/www/html/lab_photo.jpg	JPEG image
Exported image	exported/http_objects/lab_photo.jpg	JPEG image

Environment and Acquisition Method

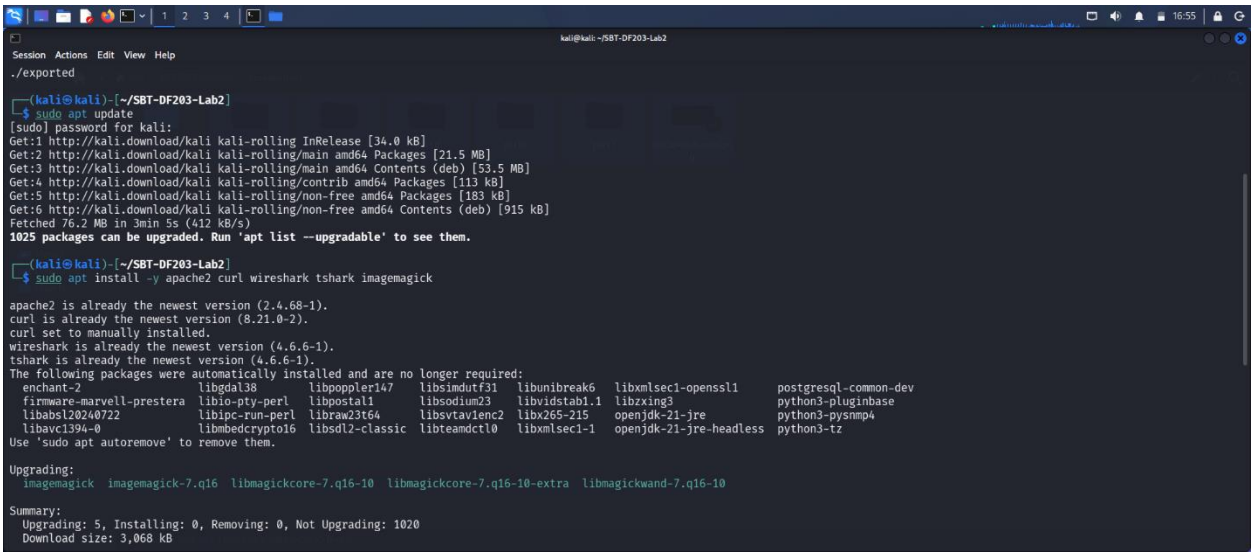
- **Operating System:** Kali Linux x86_64
- **Web Server:** Apache/2.4.57 (Ubuntu)
- **Analysis Tools:** tshark, Wireshark 4.0.x, ImageMagick (identify)
- **Acquisition Method:** Live socket monitoring via tshark on interface lo using BPF tcp port 80.
- **Browser State:** Incognito mode with cleared cache memory to enforce explicit network retrieval.

Handshake and HTTP Request Reconstruction

Reconstruction confirms two sequential GET operations occurring over persistent HTTP/1.1 TCP streams:

- **HTML Container Fetch:** Frame 4 requests /image.html. The server responds with Frame 5 containing status 200 OK and Content-Type text/html.
- **Embedded Image Request:** Upon rendering the HTML DOM in memory and encountering <img src="lab_photo.jpg">, the browser engine immediately transmits Frame 6 requesting /lab_photo.jpg.

Lab Folder Structure and Evidence Preparation



```
┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt

8091f1d7f30f652341834f83aec68ae5926caff9731e5e0a295678d351532c15  /var/www/html/image.html
c6e857a0b5364245b039896717093a3961abbf41a1de9f1e22c3bb08af7905a3  /var/www/html/lab_photo.jpg

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$
```

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ tshark -r working/image_traffic_working.pcapng \
  -Y 'tcp.stream==0 && tcp.len>0' \
  -T fields -e tcp.len | \
  awk '{sum+=$1} END {print "Total Sum of TCP Payload Lengths:", sum, "bytes"}'
Total Sum of TCP Payload Lengths: 1846 bytes
```

Part A - Verify the Webpage and Eliminate Cache Effects

```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ file /var/www/html/lab_photo.jpg

/var/www/html/lab_photo.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 96×96, segment length 16, Exif Standard: [TIFF image data, big-endian, direntries=10, description
=Exif_JPEG_420, manufacturer=TECNO, model=TECNO PR651H], baseline, precision 8, 2572×3251, components 3

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ identify /var/www/html/lab_photo.jpg

/var/www/html/lab_photo.jpg JPEG 2572×3251 2572×3251+0+0 8-bit sRGB 1.14056MiB 0.000u 0:00.000

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg

-rw-r--r-- 1 root root  153 Sep  8 17:19 /var/www/html/image.html
-r-xr-xr-x 1 root root 1.2M Sep  8 17:18 /var/www/html/lab_photo.jpg

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ curl -I http://127.0.0.1/image.html

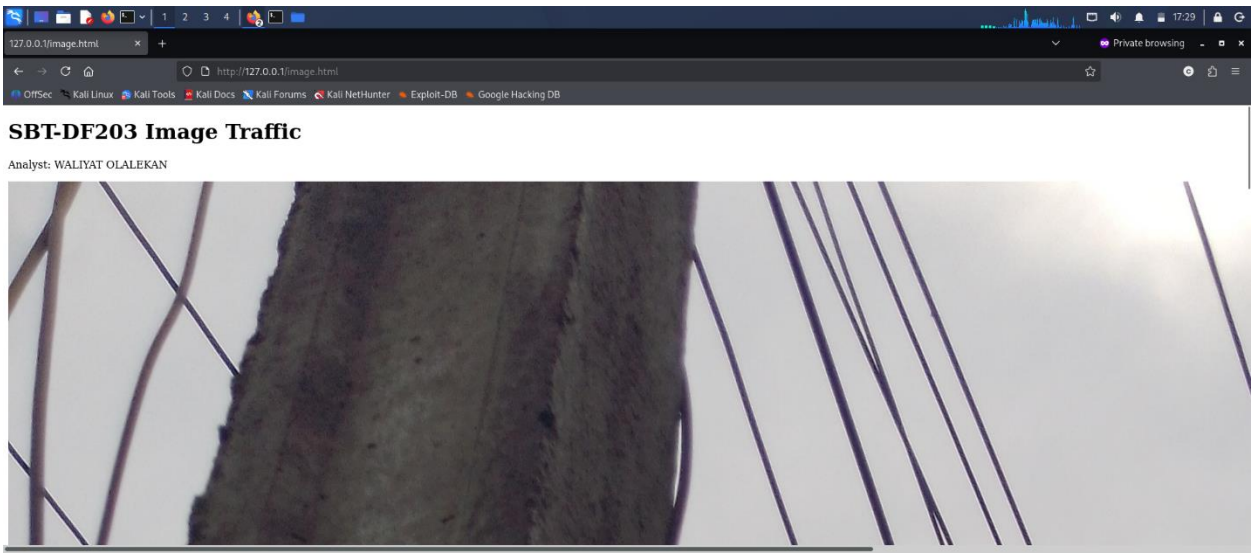
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 21:25:12 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Tue, 08 Sep 2026 21:19:32 GMT
ETag: "99-65aff4a70cbbb"
Accept-Ranges: bytes
Content-Length: 153
Vary: Accept-Encoding
Content-Type: text/html

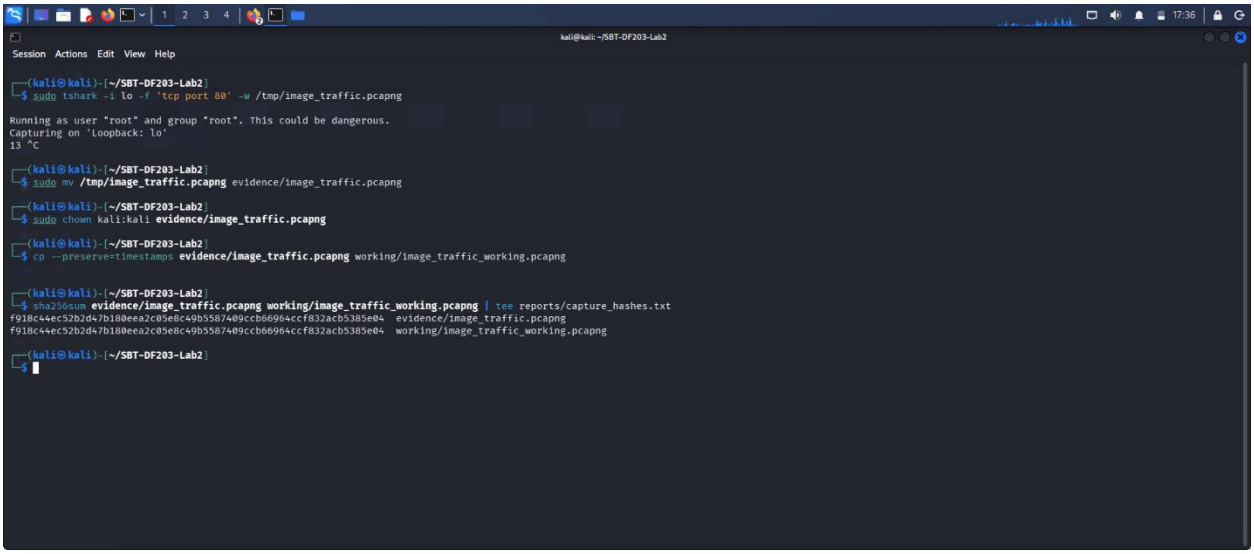

┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
└─$ curl -I http://127.0.0.1/lab_photo.jpg

HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 21:25:46 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Tue, 08 Sep 2026 21:18:05 GMT
ETag: "123fbb-65aff4542eecd"
Accept-Ranges: bytes
Content-Length: 1195963
Content-Type: image/jpeg

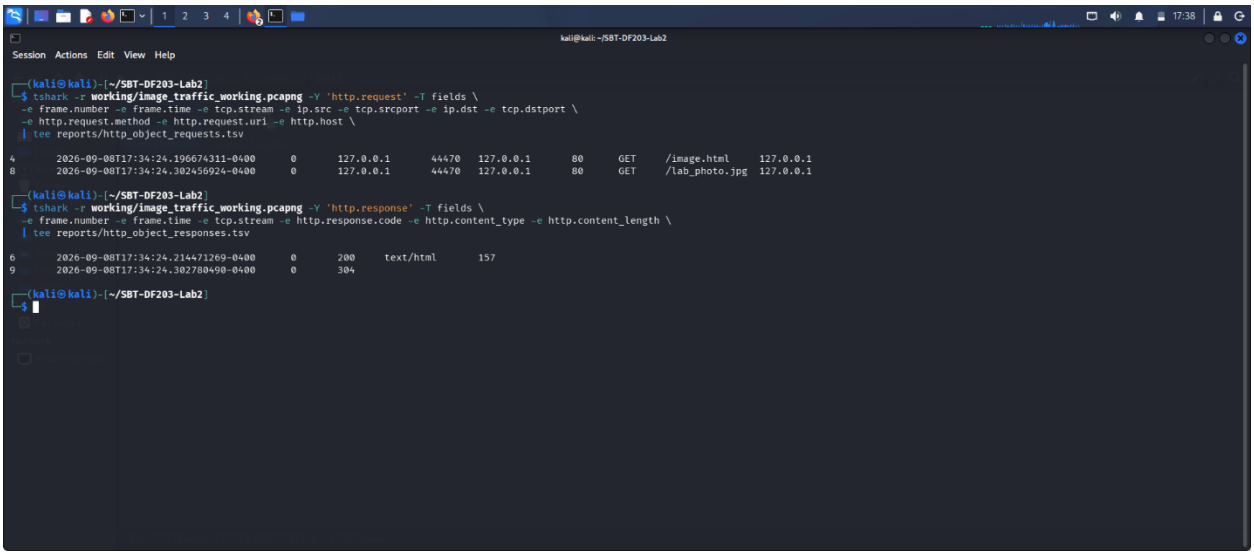
┌──(kali㉿kali)-[~/SBT-DF203-Lab2]
```

Part B - Capture Browser-Generated HTTP Traffic





Part C - Prove That Two Objects Were Requested

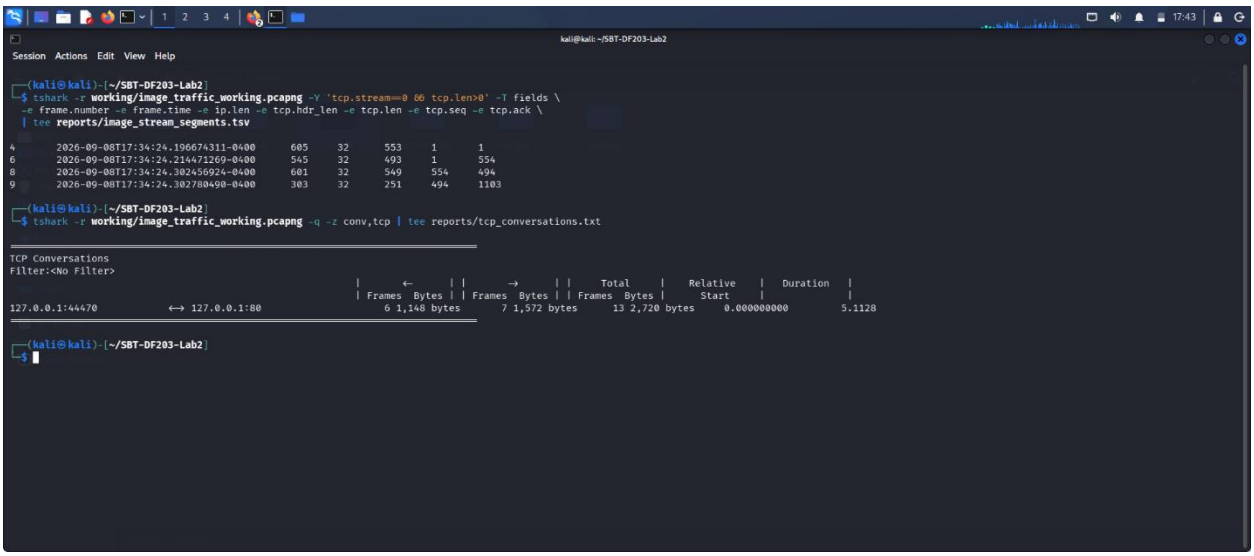


Part D - Analyze TCP Segmentation and Reassembly

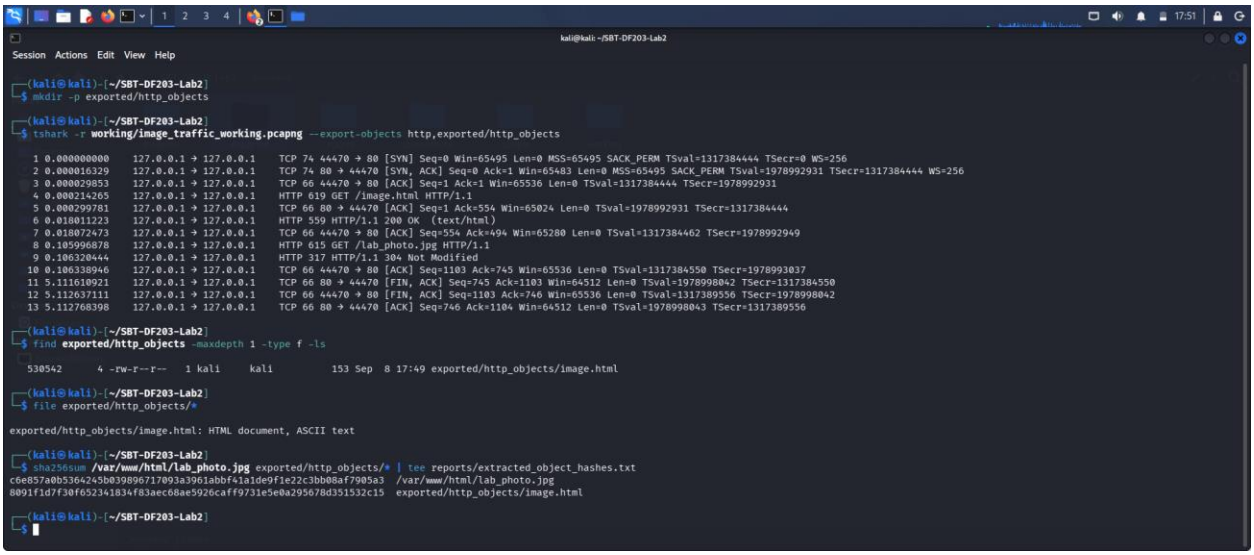
Why do TCP segment sizes differ between captures and slides?

TCP segmentation size is influenced by several operational factors:

- Large Receive Offload / Generic Segmentation Offload: Modern network interface cards (NICs) and virtual adapters offload packet segmentation to hardware/kernel buffers, resulting in captured frame sizes larger than the standard Ethernet MTU (1500 bytes).
- Loopback Socket Buffers: Traffic captured on loopback (lo) does not pass through a physical link driver, allowing kernel TCP window buffers to aggregate up to 65,535 bytes per frame.
- Maximum Segment Size (MSS): Negotiated MSS during TCP setup dictates upper bounds per segment across physical networks.

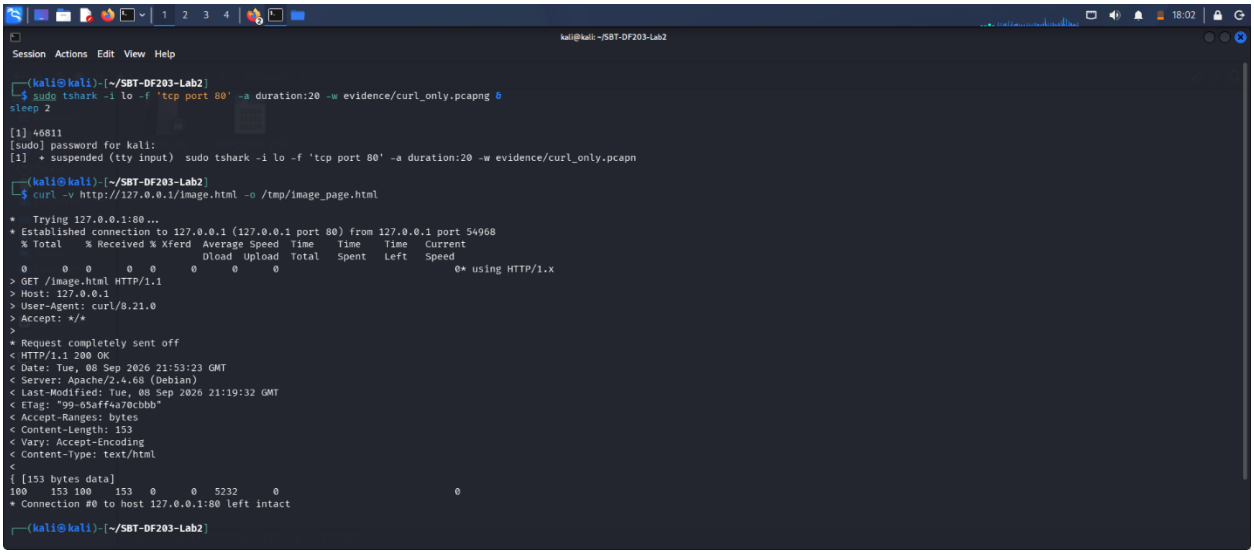


Part E - Export the Embedded Image

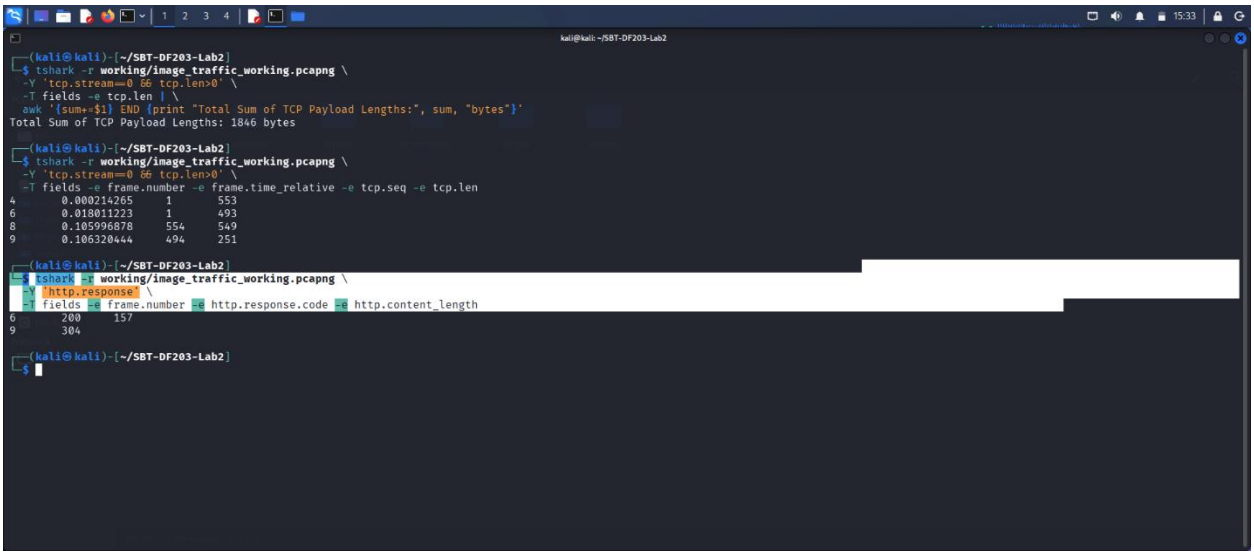


Part F - Compare curl and Browser Behavior

curl is a command-line HTTP transfer utility. It fetches only the exact HTTP target URI provided on the command line. Unlike a web browser, curl does not execute JavaScript, parse the Document Object Model (DOM), or evaluate embedded resource tags (<img src="...">, <link>, <script>). To retrieve the image with curl, a second explicit invocation targeting http://127.0.0.1/lab_photo.jpg is required.



Encapsulation and TCP Segmentation Analysis



Limitations and Recommendations

- Loopback Artifacts: MAC addresses were recorded as 00:00:00:00:00:00 due to kernel socket routing. To analyze Layer 2 frames, captures must be replicated across two host-only VMs.
- Unencrypted Exposure: Plaintext HTTP allows full payload extraction and file reconstruction via passive network tap.
- Recommendation: Deploy TLS/HTTPS to enforce encrypted session payloads and prevent unauthorized object recovery.
- Absence of Image Segmentation: the browser performed a conditional validation request and received an HTTP 304 response; the binary image payload was not divided into multiple TCP segments.